



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 6

Grupo 3

Tecnologías de Datos Masivos /
Big Data Technology

Table of Contents

Introducción	3
Contexto.....	3
Descripción del problema y objetivos del sprint.....	3
Justificación	3
Metodología	4
Entorno de Desarrollo	4
Pruebas realizadas	4
Resultados	5
Conclusión	7
Anexo	8

Introducción

Contexto

Una vez implementado el pipeline de ingesta y procesamiento en tiempo real en el sprint anterior, el paso siguiente es persistir los datos en un almacén que permita su consulta y visualización de forma eficiente. Elasticsearch es un motor de búsqueda y análisis distribuido especialmente adecuado para series temporales de eventos, gracias a su indexación por campos de fecha y su integración nativa con Kibana.

En este sprint se extiende la arquitectura cerrando el pipeline: los mensajes que fluyen por los topics de Kafka son consumidos y almacenados en índices de Elasticsearch, quedando disponibles para consultas en tiempo real y para su visualización en dashboards.

Descripción del problema y objetivos del sprint

El objetivo del sprint es implementar dos consumers Kafka que lean los mensajes de los topics producidos en sprints anteriores e inserten cada evento como documento en Elasticsearch, habilitando así la capa de visualización del pipeline.

- **Consumer de datos raw BTC:** lee el topic `gittba_BTC` e inserta cada vela cerrada en el índice `gittba_btc`, con los campos `symbol`, `@timestamp`, `close` y `volume`.
- **Consumer de datos VWAP:** lee el topic `gittba_BTC_VWAP` e inserta cada resultado de ventana en el índice `gittba_btc_vwap`, con los campos `window_start`, `window_end`, `symbol`, `vwap` y `@timestamp`, derivado de `window_end` para compatibilidad con Kibana.

Ambos consumers crean automáticamente el índice con mapping explícito si no existe, garantizando la tipificación correcta de los campos de fecha y los numéricos.

Justificación

Elasticsearch ofrece indexación eficiente sobre campos de tipo fecha, lo que permite consultar rangos temporales sobre grandes volúmenes de eventos con latencias muy bajas. Su integración con Kibana facilita la creación de dashboards interactivos sin necesidad de código adicional, lo que acelera la validación visual de los indicadores calculados.

El uso de la librería `requests` para las llamadas a la API REST de Elasticsearch simplifica la integración al eliminar dependencias de clientes de alto nivel, manteniendo el código ligero y explícito. La separación en dos consumers independientes permite escalar o modificar cada flujo de forma autónoma.

Metodología

Entorno de Desarrollo

Los dos consumers se han desarrollado en Python y ejecutado sobre el clúster del entorno de prácticas del ICAI. Elasticsearch está disponible en <http://192.168.80.37:9201> con autenticación básica, y los topics Kafka del sprint anterior siguen activos en el broker 192.168.80.34:9092.

Los componentes utilizados son:

- **kafka-python:** consumo de los topics **gittba_BTC** y **gittba_BTC_VWAP** con asignación manual de partición y `auto_offset_reset=earliest`.
- **Elasticsearch 8.x** con autenticación HTTP básica, accesible mediante la librería `requests`. Los índices se crean con mapping explícito antes del primer documento.
- **Kibana:** visualización de los índices sobre el mismo clúster, usada para verificar la correcta indexación y explorar los datos.

Pruebas realizadas

Las pruebas se realizaron con el pipeline completo activo: el producer de la HU-6 publicando velas en tiempo real y el job de Spark Structured Streaming de la HU-7 calculando el VWAP. Sobre ese flujo se lanzaron ambos consumers de forma simultánea.

Para el consumer de datos raw, se verificó que cada vela cerrada recibida del topic `gittba_BTC` generaba un nuevo documento en el índice `gittba_btc`, con el campo `@timestamp` correctamente parseado como tipo `date`. La consola mostró el identificador de documento asignado por Elasticsearch, el símbolo, el timestamp y el precio de cierre.

Para el consumer de VWAP, se verificó que cada resultado de ventana recibido del topic `gittba_BTC_VWAP` generaba un documento en el índice `gittba_btc_vwap`. Se comprobó que el campo `@timestamp` se derivaba correctamente de `window_end`, normalizando el formato emitido por Spark al estándar UTC ISO 8601 requerido por Elasticsearch.

```
(venv) gonzaloalonsolidon@Gonzalos-MacBook-Pro spark % pip install kafka-python requests
python3 elastic_consumer_btc.py
537
[OK] Insertado en ES | id=toG5q50BZBoeEvh9SRKB | symbol=BTCUSDT | @timestamp=2026-04-20T16:28:59Z | close=75396.77 | volume=29.26155
[OK] Insertado en ES | id=u4G6q50BZBoeEvh9NBIy | symbol=BTCUSDT | @timestamp=2026-04-20T16:29:59Z | close=75359.76 | volume=5.20036
[OK] Insertado en ES | id=voG7q50BZBoeEvh9HhI- | symbol=BTCUSDT | @timestamp=2026-04-20T16:30:59Z | close=75356.1 | volume=6.82488
[OK] Insertado en ES | id=w4G8q50BZBoeEvh9CBLN | symbol=BTCUSDT | @timestamp=2026-04-20T16:31:59Z | close=75407.94 | volume=10.73073
█
```

```
(venv) gonzaloalonsolidon@Gonzalos-MacBook-Pro spark % python3 elastic_consumer_vwap_btc.py
Indice 'gittba_btc_vwap' creado correctamente.

Escuchando topic 'gittba_BTC_VWAP' e insertando en ES indice 'gittba_btc_vwap'...

[OK] Insertado en ES | id=f4Gtq50BZBoeEvh9bxJL | symbol=BTCUSDT | @timestamp=2026-04-20T16:15:00Z | vwap=75414.38261715646
[OK] Insertado en ES | id=LYGyq50BZBoeEvh9AhJ7 | symbol=BTCUSDT | @timestamp=2026-04-20T16:20:00Z | vwap=75389.00515191817
[OK] Insertado en ES | id=q4G2q50BZBoeEvh9lhIk | symbol=BTCUSDT | @timestamp=2026-04-20T16:25:00Z | vwap=75438.8526643331
[OK] Insertado en ES | id=wIG7q50BZBoeEvh9KBLj | symbol=BTCUSDT | @timestamp=2026-04-20T16:30:00Z | vwap=75365.72457819016
█
```

Resultados

Ambos consumers funcionaron correctamente, insertando los documentos en los índices de Elasticsearch con el mapping y los tipos de campo esperados. La creación automática del índice con mapping explícito garantizó que @timestamp sea tratado como tipo date, condición necesaria para que Kibana pueda construir visualizaciones temporales sobre los datos.

Con los índices poblados fue posible acceder a Kibana y explorar los datos de cotización raw y los valores VWAP a través de dashboards, cerrando el pipeline end-to-end desde Binance hasta la capa de visualización.



Conclusión

En este sprint se ha completado la capa de persistencia del pipeline de datos en tiempo real, implementando dos consumers Kafka que almacenan de forma continua los eventos de cotización y los valores VWAP en índices de Elasticsearch. La creación automática de los índices con mapping explícito garantiza la correcta tipificación de los campos temporales y numéricos desde el primer documento insertado.

El pipeline completo queda así cerrado de extremo a extremo: desde la captura de datos en tiempo real desde Binance, pasando por la transmisión mediante Kafka y el cálculo del VWAP con Spark Structured Streaming, hasta la persistencia en Elasticsearch y su visualización en Kibana. La arquitectura modular adoptada, con producers y consumers independientes para cada flujo, facilita la incorporación de nuevos indicadores o fuentes de datos en futuras iteraciones.

Anexo

Enlace a GitHub: <https://github.com/GonAlonsoLid/trading-view-big-data>